Archipelago MVP: Final Plan

XRPL Hackathon, Agentic Finance track. Updated Oct 22, 2026.

Overview

The MVP is one vertical slice: a single USD to SGD payment that an AI agent routes, a policy engine gates, XRPL settles, and an audit hash verifies.

Built in 36 hours by 5 people (PM, two tech leads, QA, Security). It simulates Swift-style bank tokenized deposits on XRPL testnet: no real money, and no claimed Swift integration.

Stack

- **Build:** TypeScript and xrpl.js, starting from the open-source Scaffold-XRP template (Next.js, React). The API and dashboard extend it.
- **Agent:** a Claude API tool-use loop with 4 in-process tools. Haiku 5.5 while building, Sonnet 5.5 for the demo, behind one swappable function.
- **Data:** SQLite for audit records. QuickNode Streams feed the ETL pipeline.
- **API:** about 5 endpoints: Stream receiver, dashboard calls, approve button, kill switch.

Pitch: Swift built the connector for banks. We built the agent that decides where liquidity should flow between the islands.

Scope

The MVP keeps the smallest set of parts that still gives a working end-to-end payment.

Area	In the MVP	Cut
Islands	BankNY-USD, BankSG-SGD, XRP as the bridge. RLUSD (Ripple's USD stablecoin) as the second USD island, once a test offer on it clears	GBP, BankTX-USD, mock external island
Liquidity	Order books only	AMM pools
Routes	2 to 3 (direct, via XRP), slippage in bps	4+ routes with fee and time scoring
Gap filling	None in the core build	Auto-execution on approval
Policy	Allowlist, max amount, max slippage, approval threshold, kill switch	Daily caps, per-island buffers
Extras	None	ISO 20022 layer, x402, lending, vaults, yield
Reality	XRPL testnet, simulated issuers	Real Swift ledger, real KYC, production security

The six build pieces

The agent only calls tools. It never computes amounts and never touches keys; every transaction passes the policy engine before the signer sees it.

#	Piece	What it does	Done by
1	Seed and reset	Creates 2 issuers plus XRP, trust lines (including RLUSD, funded from Ripple's testnet faucet), balances and order books at fixed fictional prices (1 XRP = 2.00 USD, 1 SGD = 0.74 USD). Deep USD book, thin SGD book. `reset.ts` returns it to a clean state	Code
2	Island map and route finder	Reads balances (`account_lines`) and depth (`book_offers`), then computes 2 to 3 routes with slippage in bps	Code
3	Policy engine	YAML limits plus a small TypeScript check that returns approve, block (with reason code) or needs_human. It includes a counterparty-risk check behind one interface: a mock at first, swapped for Mastercard if mentors confirm an API	Code
4	Executor and audit	Payment with `SendMax` and `DeliverMin`. An in-process signer refuses anything without an approval. SHA-256 of the decision record goes in the memo; the full record goes in SQLite	Code
5	Agent	One Claude tool-use loop with 4 tools: `get_island_map`, `find_routes`, `preflight`, `request_execution`. Picks among computed routes and explains the choice	LLM chooses and explains
6	Dashboard	One Next.js page: island map, ranked routes, policy decision, and the tx hash with a hash-match check	Code

QuickNode data pipeline

QuickNode Streams feed the ETL pipeline, using the QuickNode tokens provided for the hackathon, which include testnet. Streams are not on the critical path: the island map and route finder query XRPL RPC directly for fresher state.

1. **Events:** Payment, OfferCreate and OfferCancel on the demo issuers, treasury and executor accounts, filtered by a Streams JavaScript function.
2. **Destination:** the Stream posts to our API, which writes to SQLite. QuickNode needs a public URL, so local development uses a tunnel.
3. **Readers:** the audit tab and an activity feed read from it. The island map does not.

Test before hackathon

These are throwaway checks to remove surprises before Saturday. Do not commit them to the project repo, because only work built during the hackathon counts toward judging.

- Create a QuickNode Stream on XRPL testnet with the provided tokens and confirm it is available.

- Fund two faucet wallets, send one test payment, and confirm the event reaches a test endpoint.
- Write a small Streams JavaScript filter that keeps only Payment, OfferCreate and OfferCancel for one account.
- Confirm a tunnel gives QuickNode a public URL that reaches a local endpoint.
- Note the faucet amounts, rate limits and QuickNode token usage so seeding can be sized.
- Set up a testnet RLUSD trust line, get RLUSD from the faucet, post one RLUSD/XRP offer, and confirm a payment path through it clears.

Team ownership

Tech lead 1 owns the critical path (seed and routes), so give that person the least outside work.

Role	Owns
Tech lead 1	Seed, `reset.ts`, XRPL client, island map, route finder, executor
Tech lead 2	Policy engine, signer, audit, agent loop, API
QA	Verifies `reset.ts`; adversarial policy tests (unlisted address, oversized amount, excess slippage, kill switch); audit hash round-trip; end-to-end run. Starts at hour 2 so tests are ready as each piece lands
Security	Reviews the policy engine, signer and key handling (scope to confirm)
PM	Next.js dashboard (built with Claude Code or a free AI coding tool), `config.yaml`, README, mentor questions, Jira board

36-hour timeline

Hours	Phase
0 to 2	Kickoff: mentor check and repo
2 to 10	Seed and reset
10	Checkpoint: first cross-currency payment works by hand
10 to 20	Core build: island map, routes, policy engine, executor, audit
20 to 28	Agent loop and dashboard
28 to 36	Fixes, rehearsal and README

Seed and reset must pass the hour-10 checkpoint, or the plan drops to one order book and 2 routes.

Cut order if behind

Never cut the seed, route finder, policy engine or executor. Cut in this order:

1. The RLUSD island, if its testnet book can't be made to work.

2. The approval queue: replace it with one Approve button. The policy engine still returns needs_human.
3. The audit verify screen: keep writing the hash to the memo, and log the match instead of showing it.

If seeding is not working by hour 10, drop to one order book and 2 routes.

Stretch goals

Only after the core demo works. Suggested order:

1. **AutoSplit:** one natural-language instruction splits a payout across wallets, reusing the agent, policy engine and executor.
2. **Mastercard:** swap the mock counterparty-risk check for a real one, if mentors confirm an API.
3. **Ledger:** the risk head confirms large approvals on a hardware device, if mentors confirm support.
4. **AMM pools** and automatic gap filling.
5. **GBP island** and the ISO 20022 layer.

Optional: Base44 for the frontend, to use builder credits. It needs Next.js setup and is not required.

Risks and open questions

Seed state is the biggest risk: testnet has no natural liquidity, so the books have to be built from faucet funds.

- **Mentor questions at hour 0:** is AMM or any other needed feature enabled on testnet, what are the faucet amounts and rate limits, and is a possible Mastercard or Ledger integration (a sponsor track, API or sandbox) available?
- **LLM provider:** the 10/19 notes expect AWS Bedrock may be required or favored, while the brief uses the Anthropic API. Keep the LLM call in one function so the provider can be swapped quickly.
- **Eligibility:** only work built during the hackathon counts toward judging, so no repo or code before Saturday kickoff. Prep is fine: read the XRPL docs for `ripple_path_find`, `DeliverMin` and trust lines.
- **Team decision:** Archipelago is the finalized project. AutoSplit is a stretch goal, and direction stays locked after Friday.
- **Honesty:** the build simulates Swift-style tokenized deposits. State that plainly in the README and pitch.